Algorithmique et initiation à la programmation

Informatique — Licence 1 (L1)

Cours INF-L1-01

Institut Africain de Management (IAM) — Bamako, Mali
Bibliothèque Universitaire — Bronbah Tech

À propos de ce cours
Apprendre à décomposer un problème et à le résoudre pas à pas avec des algorithmes écrits en pseudo-code : variables, conditions, boucles, tableaux, sous-programmes et premiers tris.

Document pédagogique original créé pour les étudiants de l'IAM : leçons détaillées, exemples commentés, travaux dirigés et pratiques, exercices corrigés, QCM d'évaluation et vidéos recommandées. Aucun contenu n'est reproduit d'ouvrages existants.

Sommaire

• Introduction à la matière
• Objectifs d'apprentissage
• Leçon 1 — Qu'est-ce qu'un algorithme : définition, propriétés et exemples du quotidien
• Leçon 2 — Variables, types de données et affectation
• Leçon 3 — Les structures conditionnelles : si, sinon, selon
• Leçon 4 — Les boucles : pour, tant que, répéter
• Leçon 5 — Les tableaux à une et deux dimensions
• Leçon 6 — Les sous-programmes : fonctions et procédures
• Leçon 7 — Initiation à la complexité et aux algorithmes de tri simples
• Travaux dirigés (TD)
• Travaux pratiques (TP)
• Exercices corrigés supplémentaires
• QCM d'évaluation
• Résumé — les notions importantes
• Vidéos associées aux leçons
• Sources et références

Comment utiliser ce cours : lis chaque leçon dans l'ordre, refais les exemples à la main, tente chaque exercice AVANT de lire son corrigé, puis termine par le QCM pour vérifier ce que tu as retenu.

Introduction à la matière

Avant d'écrire le moindre programme, un informaticien apprend à raisonner : quel est le problème posé, quelles données reçoit-on, quel résultat doit-on produire, et par quelles étapes y arriver à coup sûr ? Cette manière de penser s'appelle l'algorithmique. Elle ne dépend d'aucun langage de programmation : un algorithme correct en pseudo-code se traduira ensuite en C, en Python, en Java ou en JavaScript presque mot pour mot.

Ce cours pose les fondations de toute ta formation d'informaticien. Les variables, les conditions et les boucles que tu vas découvrir ici se retrouvent dans tous les langages et dans tous les métiers du numérique, du développement web à l'intelligence artificielle. Prends le temps de refaire chaque exemple à la main, sur papier : c'est en simulant un algorithme pas à pas que le déclic se produit.

Le cours suppose seulement les mathématiques du collège (les quatre opérations, les comparaisons, la notion de moyenne). Il se termine par les tableaux, les sous-programmes et une première idée de la complexité, qui te permettra de comprendre pourquoi certains programmes sont rapides et d'autres désespérément lents.

Objectifs d'apprentissage

À la fin de ce cours, tu seras capable de :

- Expliquer ce qu'est un algorithme et citer ses propriétés (finitude, précision, efficacité).
- Déclarer des variables, choisir leurs types et effectuer des affectations correctes.
- Traduire un raisonnement en structures conditionnelles (si, sinon, selon).
- Choisir et écrire la boucle adaptée (pour, tant que, répéter) sans boucle infinie.
- Manipuler des tableaux à une et deux dimensions (parcours, recherche, somme, maximum).
- Découper un problème en fonctions et procédures avec des paramètres bien définis.
- Comparer des algorithmes simples et réaliser un tri élémentaire (sélection, bulles).

Leçon 1 — Qu'est-ce qu'un algorithme : définition, propriétés et exemples du quotidien

Un algorithme est une suite finie d'instructions précises, écrites dans un ordre déterminé, qui permet de résoudre un problème à partir de données d'entrée pour produire un résultat. Le mot vient du nom du mathématicien persan Al-Khwârizmî, dont les travaux du IXe siècle ont posé les bases du calcul méthodique. Une recette de cuisine est un algorithme : des ingrédients (les entrées), des étapes numérotées (les instructions) et un plat (la sortie). Un mode d'emploi, un itinéraire ou la procédure pour retirer de l'argent à un distributeur sont aussi des algorithmes.

Pour mériter ce nom, une méthode doit respecter plusieurs propriétés. Elle doit être finie : elle se termine après un nombre raisonnable d'étapes. Elle doit être précise : chaque instruction est claire et sans ambiguïté, exécutable telle quelle. Elle doit être efficace : elle aboutit avec les moyens disponibles, sans détours inutiles. Enfin, elle doit être générale : elle résout une famille de problèmes, pas un seul cas — l'algorithme de la moyenne doit fonctionner pour 3 notes comme pour 300.

Un algorithme s'écrit souvent en pseudo-code, un langage intermédiaire entre le français et un vrai langage de programmation. Le pseudo-code utilise des mots-clés simples (Début, Fin, Lire, Écrire, Si, Pour…) que tout le monde comprend, sans les contraintes de syntaxe d'un langage particulier. C'est le langage de ce cours : une fois l'algorithme juste en pseudo-code, sa traduction dans n'importe quel langage devient un travail de copie.

L'essentiel
Un algorithme = des entrées, une suite finie et précise d'instructions, et une sortie. Il doit se terminer, être sans ambiguïté et résoudre une famille de problèmes, pas un cas unique.

Exemple — Calculer la moyenne de deux notes

```
Algorithme MoyenneDeuxNotes
Variables
n1, n2, moyenne : Réel
Début
Écrire("Entre la première note :")
Lire(n1)
Écrire("Entre la deuxième note :")
Lire(n2)
moyenne ← (n1 + n2) / 2
Écrire("La moyenne est : ", moyenne)
Fin
```

À retenir

- Un algorithme transforme des données d'entrée en un résultat, en un nombre fini d'étapes.
- Précis et non ambigu : quelqu'un d'autre (ou une machine) doit pouvoir l'exécuter sans deviner.
- Le pseudo-code sert à concevoir et vérifier la logique avant de coder dans un vrai langage.

Exercice

Décris, en quelques lignes de français structuré (pas de pseudo-code complet), l'algorithme du quotidien « préparer un thé » en respectant les propriétés d'un algorithme : entrées, étapes ordonnées, fin garantie. Puis explique pourquoi « faire chauffer l'eau assez longtemps » n'est PAS une instruction acceptable dans un algorithme.

✔ Corrigé

Entrées : eau, sachet de thé, tasse, bouilloire. Étapes : 1) remplir la bouilloire d'eau ; 2) brancher et allumer ; 3) attendre l'ébullition (l'arrêt automatique signale la fin de cette étape) ; 4) verser l'eau dans la tasse ; 5) plonger le sachet ; 6) attendre 3 minutes ; 7) retirer le sachet. Sortie : une tasse de thé. La procédure se termine toujours. En revanche, « assez longtemps » est ambigu : deux personnes comprendraient deux durées différentes, et une machine ne peut pas l'exécuter. Une instruction d'algorithme doit être mesurable : « attendre 3 minutes » ou « attendre l'ébullition ».

Leçon 2 — Variables, types de données et affectation

Une variable est un emplacement de la mémoire de l'ordinateur, désigné par un nom, qui contient une valeur pouvant changer pendant l'exécution. On la compare souvent à une boîte étiquetée : l'étiquette est le nom (par exemple « age »), le contenu est la valeur (par exemple 19). Déclarer une variable, c'est réserver la boîte et annoncer ce qu'elle contiendra. Le nom doit être clair et parlant : « prix_total » vaut mieux que « x » quand le programme grandit.

Chaque variable a un type, qui fixe la nature des valeurs acceptées et les opérations possibles. Les types de base sont : Entier (… −2, 0, 7, 250), Réel (3,14 ; 12,5 ; −0,75), Caractère ('A', '7', '?'), Chaîne de caractères ("Bonjour", "IAM") et Booléen (Vrai ou Faux). On ne peut pas additionner deux chaînes comme deux nombres, ni stocker 12,5 dans un entier sans perdre la partie décimale : choisir le bon type dès la déclaration évite une foule d'erreurs.

L'affectation est l'opération qui place une valeur dans une variable ; on la note avec une flèche : compteur ← 0. Attention, ce n'est pas une égalité mathématique : l'instruction x ← x + 1 est absurde en mathématiques mais parfaitement normale en algorithmique — elle signifie « prends l'ancienne valeur de x, ajoute 1, range le résultat dans x ». Pour échanger le contenu de deux variables, il faut une troisième variable temporaire : on ne verserait pas deux verres l'un dans l'autre sans un troisième verre.

L'essentiel
Une variable = un nom, un type, une valeur. L'affectation (←) range une valeur ou le résultat d'un calcul dans la variable ; elle remplace l'ancien contenu.

Exemple — Échanger deux variables et calculer un prix TTC

```
Variables
a, b, temp : Entier
prixHT, prixTTC : Réel
Début
a ← 5
b ← 9
temp ← a // temp vaut 5
a ← b // a vaut 9
b ← temp // b vaut 5 : échange réussi
prixHT ← 10 000
prixTTC ← prixHT * 1,18 // TVA de 18 %
Écrire(prixTTC) // affiche 11 800
Fin
```

À retenir

- Types de base : Entier, Réel, Caractère, Chaîne, Booléen — le type conditionne les opérations possibles.
- x ← x + 1 incrémente x : l'affectation lit l'ancienne valeur puis écrase par la nouvelle.
- Échanger deux variables exige une variable temporaire.

Exercice

On exécute : a ← 3 ; b ← a * 2 ; a ← b − a ; b ← a + b. Donne la valeur finale de a et de b, en détaillant chaque étape. Puis écris les déclarations (nom + type) adaptées pour stocker : un numéro de téléphone affiché tel quel, un prix en francs, une mention « admis » ou « ajourné », le nombre d'étudiants d'une classe.

✔ Corrigé
Étape par étape : a = 3 ; b = 3 × 2 = 6 ; a = 6 − 3 = 3 ; b = 3 + 6 = 9. Valeurs finales : a = 3, b = 9. Déclarations : telephone : Chaîne de caractères (on ne calcule pas dessus et il peut commencer par un indicatif ou contenir des espaces) ; prix : Réel (ou Entier si l'on ne garde que des francs entiers — les deux se défendent, Réel est plus sûr) ; mention : Chaîne (« admis »/« ajourné ») — un Booléen admis suffirait si l'on ne veut que deux cas ; nbEtudiants : Entier.

Leçon 3 — Les structures conditionnelles : si, sinon, selon

Jusqu'ici, nos algorithmes exécutaient leurs instructions toujours dans le même ordre. La structure conditionnelle permet de choisir : SI une condition est vraie, on exécute un bloc d'instructions ; SINON, on en exécute un autre (ou rien). La condition est une expression booléenne construite avec des comparaisons (=, ≠, <, >, ≤, ≥) et des connecteurs logiques : ET (les deux doivent être vrais), OU (au moins un), NON (inverse). Par exemple, « age ≥ 18 ET inscrit = Vrai » décide d'un droit d'accès.

Quand plus de deux cas sont possibles, on enchaîne les conditions : SI… SINON SI… SINON, ou on utilise la structure SELON, plus lisible quand on compare une même variable à plusieurs valeurs distinctes (un numéro de jour, un code de mention). Attention à l'ordre des tests dans une cascade : on teste d'abord le cas le plus précis ou le plus exigeant, car dès qu'une condition est vraie, les suivantes sont ignorées.

L'erreur classique du débutant est de confondre l'affectation (←) et le test d'égalité (= dans une condition). Autre piège : les conditions impossibles ou jamais vraies, comme « note < 0 ET note > 20 », qui signalent en général une erreur de logique — presque toujours, c'est un OU qu'il fallait écrire. Avant de coder, formule la condition en français : si la phrase est bancale, l'algorithme le sera aussi.

L'essentiel
SI condition ALORS … SINON … permet à l'algorithme de décider. Les conditions combinent comparaisons et connecteurs ET / OU / NON ; leur exactitude fait toute la fiabilité du programme.

Exemple — Mention selon la moyenne

```
Variables
moyenne : Réel
Début
Lire(moyenne)
Si moyenne ≥ 16 Alors
Écrire("Mention Très Bien")
Sinon Si moyenne ≥ 14 Alors
Écrire("Mention Bien")
Sinon Si moyenne ≥ 12 Alors
Écrire("Mention Assez Bien")
Sinon Si moyenne ≥ 10 Alors
Écrire("Mention Passable – admis")
Sinon
Écrire("Ajourné")
FinSi
Fin
```

À retenir

- Une condition s'évalue toujours en Vrai ou Faux : c'est elle qui aiguille l'exécution.
- Cascade SI / SINON SI : la première condition vraie l'emporte, l'ordre des tests compte.
- SELON est plus lisible quand on compare une variable à des valeurs précises.

Exercice

Écris l'algorithme qui lit l'âge d'une personne et affiche sa catégorie au cinéma : « Enfant » si moins de 12 ans, « Jeune » de 12 à 25 ans, « Adulte » au-delà. Puis dis ce qu'affiche ton algorithme pour 12 ans et pour 25 ans, et vérifie qu'aucun âge ne tombe « entre deux catégories ».

✔ Corrigé
SI age < 12 ALORS « Enfant » SINON SI age ≤ 25 ALORS « Jeune » SINON « Adulte ». Pour 12 ans : le premier test est faux (12 n'est pas < 12), le second est vrai → « Jeune », ce qui est correct puisque la catégorie commence à 12. Pour 25 ans : « Jeune » également (25 ≤ 25). Les trois intervalles [< 12], [12–25], [> 25] couvrent tous les âges sans trou ni chevauchement : la vérification aux bornes (11, 12, 25, 26) est le réflexe à garder pour toutes les cascades.

Leçon 4 — Les boucles : pour, tant que, répéter

Une boucle répète un bloc d'instructions. La boucle POUR s'utilise quand on connaît à l'avance le nombre de répétitions : « Pour i de 1 à 10 » exécute le bloc dix fois, le compteur i prenant successivement chaque valeur. C'est l'outil naturel pour afficher une table de multiplication, remplir un tableau ou répéter une saisie un nombre fixe de fois.

La boucle TANT QUE répète pendant qu'une condition reste vraie ; le nombre de tours n'est pas connu à l'avance (« Tant que le solde est insuffisant, redemander le montant »). La boucle RÉPÉTER…JUSQU'À fonctionne à l'inverse : elle exécute le bloc au moins une fois, puis recommence jusqu'à ce que la condition de sortie devienne vraie —

parfaite pour valider une saisie (« répéter la demande jusqu'à obtenir une note entre 0 et 20 »).

Le danger mortel des boucles est la boucle infinie : si la condition ne devient jamais fausse, le programme ne se termine plus. La parade tient en une phrase : à chaque tour, quelque chose doit rapprocher de la condition de sortie — un compteur qui augmente, une valeur qui diminue, une saisie nouvelle. Avant d'écrire une boucle, demande-toi toujours : « qu'est-ce qui change à chaque tour, et pourquoi cela finira-t-il par s'arrêter ? »

L'essentiel
POUR quand le nombre de tours est connu ; TANT QUE quand on répète sous condition (0 tour possible) ; RÉPÉTER quand il faut au moins un tour. Chaque boucle doit contenir ce qui la fera s'arrêter.

Exemple — Somme des entiers de 1 à n (boucle POUR) et validation de saisie (RÉPÉTER)

```
Variables
n, i, somme, note : Entier // (note : Réel si décimales)
Début
Lire(n)
somme ← 0
Pour i de 1 à n Faire
somme ← somme + i
FinPour
Écrire("Somme de 1 à ", n, " = ", somme)
Répéter
Écrire("Entre une note entre 0 et 20 :")
Lire(note)
JusquÀ (note ≥ 0 ET note ≤ 20)
Fin
```

À retenir

- Initialiser l'accumulateur (somme ← 0) AVANT la boucle, jamais dedans.
- TANT QUE peut ne jamais entrer dans la boucle ; RÉPÉTER y entre toujours au moins une fois.
- Une boucle qui ne modifie pas sa condition de sortie est une boucle infinie.

Exercice

1) Écris l'algorithme qui calcule la factorielle d'un nombre n (n! = 1 × 2 × … × n) avec une boucle POUR, en initialisant correctement l'accumulateur. 2) Combien vaut 5! ? 3) Que se passe-t-il si l'on initialise l'accumulateur à 0 ?

✔ Corrigé
fact ← 1 ; POUR i DE 1 À n FAIRE fact ← fact × i. Pour n = 5 : 1 × 1 × 2 × 3 × 4 × 5 = 120. Si l'accumulateur part de 0, tous les produits valent 0 × i = 0 : le résultat final est 0, faux pour tout n. C'est l'inverse d'une somme (accumulateur à 0 car 0 est l'élément neutre de l'addition) : pour un produit, l'élément neutre est 1. Toujours se demander « par quoi commencer pour que le premier tour soit juste ? »

Leçon 5 — Les tableaux à une et deux dimensions

Un tableau regroupe, sous un seul nom, plusieurs valeurs du même type, rangées dans des cases numérotées. Les numéros s'appellent les indices ; selon les conventions, le premier indice est 0 ou 1 — dans ce cours, nous commençons à 1. Un tableau « notes » de 30 cases stocke les notes d'une classe entière : notes[1] est la première note, notes[30] la dernière. Déclarer le tableau, c'est réserver toutes ses cases d'un coup : notes : Tableau[1..30] de Réel.

Presque tous les traitements sur les tableaux suivent le même schéma : une boucle POUR parcourt les indices un à un et applique le même traitement à chaque case. Remplir le tableau (lire chaque case), l'afficher, calculer sa somme puis sa moyenne, chercher son maximum, ou compter les éléments qui vérifient une propriété (les notes ≥ 10, par exemple) : ce sont toujours des variations du même parcours. Celui qui maîtrise le parcours de tableau maîtrise la moitié de l'algorithmique de première année.

Un tableau à deux dimensions (une matrice) possède deux indices : la ligne et la colonne, comme une grille ou un emploi du temps — semaine[jour][heure]. On le parcourt avec deux boucles imbriquées : la boucle extérieure avance dans les lignes, la boucle intérieure dans les colonnes de chaque ligne. Pour une grille de 5 lignes et 6 colonnes, l'intérieur s'exécute 5 × 6 = 30 fois : le produit des deux tailles donne le nombre total de cases.

L'essentiel
Un tableau = une collection ordonnée de valeurs du même type, accessibles par indice. Le parcours par boucle POUR est le patron de presque tous les traitements ; deux indices et deux boucles imbriquées pour les matrices.

Exemple — Moyenne et maximum des notes d'une classe

```
Variables
notes : Tableau[1..30] de Réel
i : Entier
somme, moyenne, maxi : Réel
Début
Pour i de 1 à 30 Faire
Lire(notes[i])
FinPour
somme ← 0
maxi ← notes[1]
Pour i de 1 à 30 Faire
somme ← somme + notes[i]
Si notes[i] > maxi Alors
maxi ← notes[i]
FinSi
FinPour
moyenne ← somme / 30
Écrire("Moyenne : ", moyenne, " — Max : ", maxi)
Fin
```

À retenir

• Un indice hors limites (notes[31] dans un tableau de 30) est une erreur classique : toujours borner ses boucles à la taille réelle.

• Le maximum s'initialise avec la première case (notes[1]), pas avec 0 — sinon un tableau de valeurs négatives donnerait un faux maximum.

• Imbriquer deux boucles = traiter lignes × colonnes ; le nombre d'exécutions est le produit des tailles.

Exercice

Écris l'algorithme qui, pour un tableau de 20 températures déjà rempli, compte combien de jours la température a dépassé 35 degrés, puis affiche ce compteur. Explique pourquoi le compteur doit être initialisé avant la boucle et non dedans.

✔ Corrigé
compteur ← 0 ; POUR i DE 1 À 20 FAIRE SI temp[i] > 35 ALORS compteur ← compteur + 1. Afficher(compteur). Initialisé avant la boucle, le compteur cumule les jours sur tout le parcours. Initialisé dans la boucle, il serait remis à zéro à chaque tour : à la fin, il vaudrait au mieux 1 (si le dernier jour dépasse 35) — le programme « marcherait » en apparence tout en donnant presque toujours un résultat faux. C'est la famille d'erreurs la plus fréquente chez les débutants : le bon réflexe est de se demander, pour chaque variable, « sa valeur doit-elle survivre d'un tour à l'autre ? »

Leçon 6 — Les sous-programmes : fonctions et procédures

Quand un traitement revient plusieurs fois dans un algorithme — calculer un prix TTC, vérifier qu'une note est valide, afficher une ligne de tableau — on l'écrit une seule fois dans un sous-programme, puis on l'appelle là où on en a besoin. On évite ainsi les

copier-coller, sources d'erreurs : une correction faite dans le sous-programme bénéficie immédiatement à tous ses appels.

On distingue deux sortes de sous-programmes. La fonction reçoit des paramètres et rend un résultat : Fonction carre(x : Réel) : Réel — l'appel carre(5) vaut 25 et s'utilise dans un calcul ou une condition. La procédure reçoit des paramètres et accomplit une action sans rendre de valeur : afficher un menu, échanger deux éléments d'un tableau. Les paramètres sont les « entrées officielles » du sous-programme : ce sont des copies ou des références aux valeurs de l'appelant, définies dans l'en-tête.

Les variables déclarées à l'intérieur d'un sous-programme sont locales : elles naissent à l'appel et disparaissent à la fin, invisibles du reste du programme. Cette étanchéité est une protection — deux sous-programmes peuvent utiliser le nom « i » sans se gêner. Les variables globales, visibles partout, doivent rester rares : un programme dont toutes les parties touchent les mêmes variables devient vite impossible à corriger. Découper un gros problème en sous-programmes courts et clairs est la compétence qui sépare le bricoleur du développeur.

L'essentiel
Une fonction calcule et rend une valeur ; une procédure exécute une action. Les paramètres sont les entrées du sous-programme, ses variables internes sont locales et protégées. Découper = simplifier, réutiliser, corriger une seule fois.

Exemple — Fonction maximum et procédure d'affichage

```
Fonction maximum(a : Réel, b : Réel) : Réel
Début
Si a ≥ b Alors
Retourner a
Sinon
Retourner b
FinSi
FinFonction

Procédure afficherLigne(mot : Chaîne, n : Entier)
Variables
i : Entier
Début
Pour i de 1 à n Faire
Écrire(mot, " ")
FinPour
FinProcédure

// Appels : m ← maximum(12,5 ; 9,8) puis afficherLigne("*", 10)
```

À retenir

- Fonction = je rends un résultat ; procédure = je fais une action.
- Ce qui entre dans un sous-programme passe par ses paramètres, pas par des variables partagées en douce.
- Un sous-programme doit tenir en quelques lignes et faire UNE chose bien.

Exercice

Écris la fonction estAdmis(moyenne : Réel) : Booléen qui rend Vrai si la moyenne est au moins 10. Puis écris le fragment de programme principal qui lit la moyenne d'un étudiant et affiche « Admis » ou « Ajourné » en utilisant cette fonction.

✔ Corrigé
Fonction estAdmis(moyenne : Réel) : Booléen — Début — Retourner (moyenne ≥ 10) — FinFonction. Programme principal : Lire(m) ; SI estAdmis(m) ALORS Écrire("Admis") SINON Écrire("Ajourné"). La comparaison moyenne ≥ 10 est déjà un booléen : on peut la retourner directement, sans SI interne. Si un jour le seuil change (par exemple 10,5), une seule ligne est à modifier, quel que soit le nombre d'endroits où la fonction est appelée : c'est exactement le bénéfice recherché.

Leçon 7 — Initiation à la complexité et aux algorithmes de tri simples

Deux algorithmes peuvent résoudre le même problème avec des temps très différents. La complexité estime comment le nombre d'opérations grandit avec la taille des données. Rechercher un nom dans une liste non triée oblige, au pire, à tout parcourir : pour 1 000 noms, jusqu'à 1 000 comparaisons ; pour 1 000 000, jusqu'à 1 000 000 — on dit que c'est un coût « linéaire », noté O(n). L'idée à retenir : un bon algorithme ne se juge pas seulement à « ça marche », mais à « ça marche vite, même quand les données grossissent ».

Le tri par sélection est le tri le plus intuitif : on cherche le plus petit élément, on le met en premier ; on cherche le plus petit des restants, on le met en deuxième ; et ainsi de suite. Pour n éléments, il fait environ n × n / 2 comparaisons : on dit qu'il est en O(n²). Le tri à bulles fait remonter les grands éléments par échanges de voisins successifs ; il est aussi en O(n²), mais il a une vertu : si le tableau est déjà presque trié, il le détecte vite et s'arrête tôt dans sa version améliorée.

Ces tris simples ne servent pas dans les grands logiciels (on y utilise des tris en O(n log n), comme le tri fusion vu en L2), mais ils sont parfaits pour apprendre : ils combinent boucles imbriquées, parcours de tableau, recherche de minimum et échanges — c'est-à-dire tout le cours. Savoir exécuter un tri à la main sur un petit tableau, case par case, est d'ailleurs une question classique d'examen en L1.

L'essentiel
La complexité mesure la croissance du temps de calcul quand les données grandissent : O(n) croît doucement, O(n²) explose vite. Le tri par sélection et le tri à bulles sont les deux tris élémentaires à savoir exécuter à la main.

Exemple — Tri par sélection (ordre croissant)

```
Variables
t : Tableau[1..n] de Entier
i, j, min, temp : Entier
Début
Pour i de 1 à n − 1 Faire
min ← i
Pour j de i + 1 à n Faire
Si t[j] < t[min] Alors
min ← j
FinSi
FinPour
// échanger t[i] et t[min]
temp ← t[i]
t[i] ← t[min]
t[min] ← temp
FinPour
Fin
```

À retenir

- Juger un algorithme : correct d'abord, rapide ensuite — la complexité mesure le « rapide » quand n grandit.
- Tri par sélection : à chaque position, on place le minimum du reste du tableau.
- Deux boucles imbriquées sur n éléments ≈ n^2 opérations : à éviter sur de très gros volumes.

Exercice

Exécute le tri par sélection, étape par étape, sur le tableau [7, 3, 9, 1, 5] : donne le tableau après chaque passage de la boucle extérieure (i = 1, 2, 3, 4).

✓ Corrigé
Départ : [7, 3, 9, 1, 5]. i=1 : le minimum du tableau entier est 1 (position 4), échangé avec la position 1 → [1, 3, 9, 7, 5]. i=2 : minimum de [3, 9, 7, 5] = 3, déjà en place → [1, 3, 9, 7, 5]. i=3 : minimum de [9, 7, 5] = 5 (position 5), échangé avec la position 3 → [1, 3, 5, 7, 9]. i=4 : minimum de [7, 9] = 7, déjà en place → [1, 3, 5, 7, 9]. Le tableau est trié ; remarque qu'après le passage i, les i premières cases sont définitivement en place : c'est l'invariant qui prouve que l'algorithme est correct.

Travaux dirigés (TD)

TD 1 — Bulletin de notes complet

On donne les notes d'un étudiant dans 5 matières, avec leurs coefficients : Algorithmique 14 (coef 3), Mathématiques 9 (coef 2), Anglais 12 (coef 1), Économie 11 (coef 2), Comptabilité 8 (coef 2). Écris l'algorithme qui lit les notes et les coefficients dans deux tableaux, calcule la moyenne pondérée, puis affiche la moyenne et la décision : « Admis » si la moyenne est au moins 10, sinon « Ajourné ». Donne ensuite, à la main, la moyenne de l'exemple.

✔ Corrigé du TD 1
Moyenne pondérée = somme(note × coef) ÷ somme(coefs). Somme pondérée : 14×3 + 9×2 + 12×1 + 11×2 + 8×2 = 42 + 18 + 12 + 22 + 16 = 110. Somme des coefficients : 3 + 2 + 1 + 2 + 2 = 10. Moyenne = 110 ÷ 10 = 11,0 → Admis. En algorithme : deux tableaux notes[1..5] et coefs[1..5] ; une boucle accumule sn ← sn + notes[i] × coefs[i] et sc ← sc + coefs[i] ; moyenne ← sn / sc ; SI moyenne ≥ 10 ALORS « Admis » SINON « Ajourné ». Piège classique à éviter : diviser par 5 (le nombre de matières) au lieu de la somme des coefficients.

TD 2 — Gestion d'une petite caisse

Une boutique enregistre ses ventes de la journée dans un tableau ventes[1..n] (montants en FCFA, n demandé à l'utilisateur). Écris l'algorithme qui affiche : le total des ventes, la plus grosse vente, le nombre de ventes d'au moins 10 000 FCFA, et la moyenne des ventes. Tous les traitements doivent tenir en un seul parcours supplémentaire après le remplissage.

✔ Corrigé du TD 2
Après le remplissage : total ← 0 ; maxi ← ventes[1] ; nb ← 0 ; POUR i DE 1 À n : total ← total + ventes[i] ; SI ventes[i] > maxi ALORS maxi ← ventes[i] ; SI ventes[i] ≥ 10 000 ALORS nb ← nb + 1. Après la boucle : moyenne ← total / n ; afficher les quatre résultats. Remarque de méthode : un seul parcours suffit car les quatre traitements sont indépendants et lisent chaque case sans la modifier. Si l'on voulait en plus la plus PETITE vente, on ajouterait un mini initialisé à ventes[1] — jamais à 0 (une vente peut valoir 0, mais le raisonnement « initialiser avec la première case » reste le seul correct dans tous les cas).

Travaux pratiques (TP)

TP 1 — Simuler pas à pas et traduire en Python

Reprends l'algorithme de la moyenne et du maximum des notes (leçon 5). 1) Simule-le à la main sur le jeu de notes [12, 7, 15, 10] (en supposant le tableau déclaré pour 4 cases) et remplis un tableau de suivi des variables i, somme et maxi à chaque étape. 2) Traduis l'algorithme en Python (listes, boucle for, fonction max interdite : fais la recherche toi-même). 3) Teste ton programme avec le même jeu de notes et vérifie que tu obtiens les mêmes résultats qu'à la main.

✔ Attendus du TP 1
Simulation : après remplissage, somme = 0, maxi = notes[1] = 12. i=1 : somme = 12, maxi 12. i=2 : somme = 19, note 7 < 12, maxi 12. i=3 : somme = 34, note 15 > 12, maxi = 15. i=4 : somme = 44, maxi 15. Moyenne = 44 ÷ 4 = 11,0 ; maximum = 15. Traduction Python attendue : notes = [12, 7, 15, 10] ; somme = 0 ; maxi = notes[0] (Python numérote à partir de 0) ; boucle for v in notes : somme += v ; if v > maxi : maxi = v ; puis print(somme / len(notes), maxi). Le test doit afficher 11.0 et 15 : si tes deux méthodes (main et machine) s'accordent, tu as compris — sinon, le tableau de suivi te montre exactement à quelle étape elles divergent.

Exercices corrigés supplémentaires

Exercice 1

Écris l'algorithme qui demande un nombre entier et affiche sa table de multiplication de 1 à 10 (« 7 × 3 = 21 », etc.).

✔ Corrigé
Lire(n) ; POUR i DE 1 À 10 FAIRE Écrire(n, " × ", i, " = ", n × i). La boucle POUR est l'outil exact : le nombre de tours (10) est connu et fixe. Variante d'entraînement : affiche les tables de 1 à 10 complètes avec deux boucles imbriquées (l'extérieure pour le nombre, l'intérieure pour le multiplicateur).

Exercice 2

Écris l'algorithme qui lit un nombre entier positif et dit s'il est pair ou impair, en utilisant l'opérateur modulo (reste de la division).

✔ Corrigé
Lire(n) ; SI n MOD 2 = 0 ALORS Écrire("Pair") SINON Écrire("Impair"). Le modulo donne le reste de la division par 2 : 0 si le nombre est pair, 1 sinon. Ce même test (x MOD k) sert partout : multiples, tours de boucle alternés, répartition en groupes.

Exercice 3

Un étudiant veut savoir combien de mois il doit économiser pour acheter un ordinateur à 250 000 FCFA en mettant de côté 15 000 FCFA par mois, sans aucun autre revenu. Écris l'algorithme général (prix et économie mensuelle lus au clavier) en utilisant une boucle TANT QUE, puis donne la réponse pour l'exemple.

✔ Corrigé
total ← 0 ; mois ← 0 ; TANT QUE total < prix FAIRE total ← total + eco ; mois ← mois + 1. Afficher(mois). Pour l'exemple : 250 000 ÷ 15 000 = 16,67 → il faut 17 mois (au bout de 16 mois, total = 240 000 < 250 000 ; le 17e mois porte le total à 255 000). La boucle TANT QUE est adaptée car le nombre de tours dépend des valeurs, inconnu à l'avance. Condition de terminaison : l'économie mensuelle doit être strictement positive — un bon programme la vérifierait avant de boucler.

QCM d'évaluation

Choisis la bonne réponse pour chaque question. Les corrigés détaillés se trouvent juste après le QCM : ne les regarde qu'après avoir répondu à tout.

1. Un algorithme doit obligatoirement…

   A) Être écrit en langage C

   B) Se terminer après un nombre fini d'étapes

   C) Contenir au moins une boucle

2. Après les instructions a ← 4 ; b ← a + 3 ; a ← b, la valeur de a est…

A) 4

B) 7

C) 11

3. Le type le plus adapté pour stocker un numéro de téléphone affiché tel quel est…

A) Entier

B) Réel

C) Chaîne de caractères

4. Dans « Si moyenne ≥ 10 Alors », moyenne ≥ 10 est…

A) Une affectation

B) Une condition booléenne

C) Un type de variable

5. Quelle boucle garantit au moins une exécution de son bloc ?

A) TANT QUE

B) POUR

C) RÉPÉTER…JUSQU'À

6. On initialise l'accumulateur d'un produit (factorielle) à…

A) 0

B) 1

C) n

7. Pour parcourir un tableau de 30 cases, la boucle adaptée est…

A) TANT QUE sans compteur

B) POUR i DE 1 À 30

C) RÉPÉTER jusqu'à i = 30 sans incrémenter i

8. Le maximum d'un tableau de températures négatives, initialisé à 0, serait…

A) Toujours correct

B) Faux : il afficherait 0 qui n'est pas dans le tableau

C) Correct si le tableau est trié

9. Une fonction se distingue d'une procédure parce qu'elle…

A) Est plus rapide

B) Rend un résultat utilisable dans un calcul

C) N'a pas de paramètres

10. Un tri en $O(n^2)$ appliqué à 10 fois plus de données demande environ…

A) 10 fois plus d'opérations

B) 2 fois plus d'opérations

C) 100 fois plus d'opérations

Corrigés du QCM

1. Réponse B — La finitude est une propriété définissante : une méthode qui ne se termine pas n'est pas un algorithme utilisable.

2. Réponse B — b vaut 4 + 3 = 7, puis a reçoit la valeur de b : a = 7.

3. Réponse C — On ne calcule jamais sur un numéro de téléphone ; il peut contenir des espaces ou un indicatif : c'est une chaîne.

4. Réponse B — C'est une comparaison qui vaut Vrai ou Faux et aiguille l'exécution.

5. Réponse C — RÉPÉTER teste sa condition de sortie APRÈS le premier passage ; les deux autres peuvent faire zéro tour.

6. Réponse B — 1 est l'élément neutre de la multiplication ; partir de 0 annulerait tout le produit.

7. Réponse B — Le nombre de tours est connu (30) et le compteur sert directement d'indice.

8. Réponse B — Il faut initialiser le maximum avec la première case du tableau, pas avec une valeur arbitraire.

9. Réponse B — La fonction retourne une valeur ; la procédure exécute une action sans rien retourner.

10. Réponse C — Le coût croît comme le carré : multiplier n par 10 multiplie le temps par environ 100.

Résumé — les notions importantes

• Un algorithme est une méthode finie, précise et générale qui transforme des entrées en un résultat ; on le conçoit en pseudo-code avant de le traduire dans un langage.

• Une variable possède un nom, un type et une valeur ; l'affectation (←) remplace sa valeur, et x ← x + 1 est un incrément, pas une équation.

• Les conditions (SI / SINON / SELON) aiguillent l'exécution grâce à des expressions booléennes combinant comparaisons et ET / OU / NON.

• POUR pour un nombre de tours connu, TANT QUE pour une répétition sous condition, RÉPÉTER pour au moins un tour ; toute boucle doit progresser vers sa sortie.

• Les tableaux regroupent des valeurs de même type accessibles par indice ; somme, moyenne, maximum et comptage sont des variations d'un même parcours.

• Fonctions et procédures découpent le problème en morceaux réutilisables, avec des paramètres explicites et des variables locales protégées.

• La complexité compare les algorithmes quand les données grandissent ; sélection et bulles sont les tris élémentaires à exécuter à la main sans hésiter.

Fiches de révision express

Chaque fiche reprend les points « À retenir » d'une leçon. Relis-les la veille d'un contrôle : cinq minutes par fiche suffisent.

Fiche 1 — Qu'est-ce qu'un algorithme : définition, propriétés et exemples du quotidien
Un algorithme transforme des données d'entrée en un résultat, en un nombre fini d'étapes. ; Précis et non ambigu : quelqu'un d'autre (ou une machine) doit pouvoir l'exécuter sans deviner. ; Le pseudo-code sert à concevoir et vérifier la logique avant de coder dans un vrai langage.

Fiche 2 — Variables, types de données et affectation
Types de base : Entier, Réel, Caractère, Chaîne, Booléen — le type conditionne les opérations possibles. ; x ← x + 1 incrémente x : l'affectation lit l'ancienne valeur puis écrase par la nouvelle. ; Échanger deux variables exige une variable temporaire.

Fiche 3 — Les structures conditionnelles : si, sinon, selon
Une condition s'évalue toujours en Vrai ou Faux : c'est elle qui aiguille l'exécution. ; Cascade SI / SINON SI : la première condition vraie l'emporte, l'ordre des tests compte. ; SELON est plus lisible quand on compare une variable à des valeurs précises.

Fiche 4 — Les boucles : pour, tant que, répéter
Initialiser l'accumulateur (somme ← 0) AVANT la boucle, jamais dedans. ; TANT QUE peut ne jamais entrer dans la boucle ; RÉPÉTER y entre toujours au moins une fois. ; Une boucle qui ne modifie pas sa condition de sortie est une boucle infinie.

Fiche 5 — Les tableaux à une et deux dimensions
Un indice hors limites (notes[31] dans un tableau de 30) est une erreur classique : toujours borner ses boucles à la taille réelle. ; Le maximum s'initialise avec la première case (notes[1]), pas avec 0 — sinon un tableau de valeurs négatives donnerait un faux maximum. ; Imbriquer deux boucles = traiter lignes × colonnes ; le nombre d'exécutions est le produit des tailles.

Fiche 6 — Les sous-programmes : fonctions et procédures
Fonction = je rends un résultat ; procédure = je fais une action. ; Ce qui entre dans un sous-programme passe par ses paramètres, pas par des variables partagées en douce. ; Un sous-programme doit tenir en quelques lignes et faire UNE chose bien.

Fiche 7 — Initiation à la complexité et aux algorithmes de tri simples
Juger un algorithme : correct d'abord, rapide ensuite — la complexité mesure le « rapide » quand n grandit. ; Tri par sélection : à chaque position, on place le minimum du reste du tableau. ; Deux boucles imbriquées sur n éléments ≈ n^2 opérations : à éviter sur de très gros volumes.

Vidéos associées aux leçons

Ces vidéos, en français, complètent les leçons. Liens vérifiés au moment de la publication de ce document.

Leçon 1 — Algorithmique — série complète (28 vidéos : introduction, variables, conditions, boucles, fonctions, tableaux)

Chaîne : Série de cours YouTube · Langue : Français · Niveau : Débutant

Une série entière qui suit exactement le plan de ce cours, leçon par leçon, avec des exercices sur les variables et les structures. Idéale pour revoir tout le cours en vidéo, dans l'ordre.

http://www.youtube.com/playlist?list=PLRCmz8wMtjVZThZ_jhTvrQxIY4kbIDz3v

Leçon 2 — Les bases de l'algorithmique : Variables et opérations (Cours débutant)

Chaîne : Codelia · Langue : Français · Niveau : Débutant

Ce qu'est un algorithme, le rôle des variables, les types de données et les opérations de base, avec un exemple simple expliqué étape par étape.

https://www.youtube.com/watch?v=8yULZs5FyjE

Leçon 4 — Algorithme Pour les débutants - Variable, Condition, Boucle et le Fonction

Chaîne : KinSoft Lab · Langue : Français · Niveau : Débutant

Un cours d'algorithme pour débutants qui enchaîne les quatre piliers du cours : variables, conditions, boucles et fonctions, avec des exemples guidés.

https://www.youtube.com/watch?v=--hs-bYRigM

Leçon 5 — [Algo] Les Tableaux et Matrices from scratch Part1

Chaîne : GeniusClassrooms · Langue : Français · Niveau : Débutant à intermédiaire

Les tableaux à une dimension (vecteurs) et à deux dimensions (matrices) depuis zéro, avec les fonctions et procédures appliquées aux tableaux.

https://www.youtube.com/watch?v=o5HdQBxq2A4

Leçon 7 — Algorithmes de Tri pour Debutants - Cours Complet en Francais

Chaîne : lioncode · Langue : Français · Niveau : Débutant

Tous les tris classiques visualisés (bulles, sélection, insertion, fusion, rapide) avec la notation Big O pour mesurer leur efficacité : le prolongement direct de la leçon 7.

https://www.youtube.com/watch?v=DNGv7nJzDMI

Leçon 7 — Exercices d'Algorithmique Débutant | Du "Bonjour Monde" au PGCD

Chaîne : kojo-codeur · Langue : Français · Niveau : Débutant

Dix exercices progressifs corrigés en vidéo (parité, maximum, somme de 1 à n, factorielle, table de multiplication, PGCD) : mets la vidéo en pause et cherche avant de regarder la solution.

https://www.youtube.com/watch?v=wj6QpnaaQpc

Sources et références

• Programme officiel Licence Informatique, Université IAM (Institut Africain de Management), Bamako — vérifier les intitulés d'UE auprès du secrétariat pédagogique.

• Cormen, Leiserson, Rivest & Stein — « Introduction à l'algorithmique » (Dunod) : l'ouvrage de référence mondial, pour aller plus loin après la L1.

• Documentation Python (docs.python.org/fr) — pour traduire et tester tes algorithmes en vrai code.

• Les vidéos listées dans la section « Vidéos associées aux leçons » de ce document (liens vérifiés à la publication).

• Cours et exercices d'algorithmique des bibliothèques universitaires francophones en accès libre — demande les références à jour à la bibliothèque de l'IAM.

Fin du cours — Bronbah Tech, Bibliothèque Universitaire IAM. Bon courage pour tes révisions !

Mes notes

Utilise ces pages pour recopier les définitions, refaire les calculs et noter tes questions pour le prochain cours.